Lab 6 - The Real Nav2 Stack on the Same Robot

Aim. Drive DiffBot around the walls with **Nav2**, the standard ROS 2 navigation framework, using a simulator, a robot and a `goto` tool that are all in this folder. Compare it with a hand-written planner + navigator (the separate lab *Autonomous Navigation*).

Time. 45 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (`winget install --id prefix-dev.pixi -e --source winget`), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Nav2 splits navigation into servers that are started and supervised by a **lifecycle manager** (each server moves through *unconfigured* -> *inactive* -> *active*). Nav2 offers the standard **action** `navigate_to_pose` (a hand-written navigator, as in the separate lab *Autonomous Navigation*, can offer the same one), so any client works with either.

A hand-written navigator (lab *Autonomous Navigation*)	Nav2 (here, `nav2_diffbot.yaml`)
`planner_server` + A* on an inflated grid	`planner_server` with the **NavFn** plugin on a **global costmap** (static map + inflation layer)
pure pursuit inside `navigator`	`controller_server` with **Regulated Pure Pursuit** on a **local costmap** (lidar obstacles)
"blocked / stuck" abort	`behavior_server` (spin, back up, wait) and `bt_navigator` (a *behaviour tree* that decides what to try)
a few hundred lines of Python	C++ servers configured by one YAML file

The course launch file `nav2.launch.py` starts only the four servers a goal needs, because Nav2's stock `navigation_launch.py` stalled on this Windows setup.

Procedure

1. T1 - start the simulator and Nav2. Wait for the line `Managed nodes are active` (10-90 s; a goal sent earlier is rejected).

```
pixi run ros launch diffbot_bringup nav2.launch.py rviz:=true
```

2. T2 - the Nav2 nodes.

```
pixi run ros node list --no-daemon --spin-time 5
```

3. T2 - send the goal across the room. `goto` sends the `navigate_to_pose` action goal. Nav2 plans with NavFn and follows with Regulated Pure Pursuit.

```
pixi run goto 8.5 6.5
```

4. T2 - a goal inside a wall. A hand-written navigator may reject this. Nav2's planner accepts a goal inside an obstacle if there is a free cell nearby and drives to it.

```
pixi run goto 3.1 2.0
```

Compare the final position printed by `goto` with (3.1, 2.0). Stop T1 with Ctrl+C.

```
# Ctrl+C in T1
```

Observations (recorded)

	hand-written A* + pure pursuit (lab *Autonomous Navigation*, for reference)	**Nav2 (NavFn + Regulated Pure Pursuit), measured here**
Time to start	30-40 s	10-90 s until `Managed nodes are active`
`goto 8.5 6.5`	reached, about 13.2 m driven	reached, about 14.0 m driven, 0 collisions
`goto 3.1 2.0` (inside wall)	`FAILED: no plan`	reached: robot stopped near (3.5, 2.1), the nearest free point

Verification

```
pixi run test-nav2
```

8 checks: all Nav2 lifecycle nodes become active; `navigate_to_pose` is available; the goal is accepted and succeeds; the robot ends within 0.35 m; no collision; a plan was published on `/plan`; the robot detoured around both walls. Recorded: 14.0 m driven for a 9.3 m straight line, 0 collisions, plan published 57 times.

Questions

1. A simple navigator has a global planner, a path follower and an executive. Which Nav2 server corresponds to each?
2. Why does Nav2 need *two* costmaps (global and local)? What does each one see?
3. Why can the same `goto` script drive both stacks?
4. Open `ws\src\diffbot_bringup\config\nav2_diffbot.yaml`. Find the planner plugin, the controller plugin and `robot_radius`. What would you change to use a different planner?

If it does not work

Symptom	Fix
`goto` prints "goal rejected"	Nav2 is not active yet - wait for `Managed nodes are active` in T1
Nothing happens for minutes after the `route_server` line	you started Nav2's stock `navigation_launch.py`; use `nav2.launch.py`
`ros lifecycle get` says "Node not found"	that command has no discovery wait; read the T1 log instead